Safe Autopilot

What Could Go Wrong, and How I Tested It

Godwin Pammichukwu Emefiele Jr

github.com/donpams/safe-autopilot | JSBSim + Gymnasium + PyTorch

A learned flight controller (behavior cloning of a PID baseline) with a runtime safety monitor that falls back to the PID whenever the aircraft's state leaves a validated safe envelope. Built and tested end-to-end in a Cessna 172 JSBSim simulation. Below are the five real issues found during development, each with its root cause, how it was caught, and its resolution or documented scope boundary.

0%	2–3×	Sev. 3
monitored crash rate across every stress condition tested	better final altitude tracking with the monitor engaged	fallback's reliability cliff – reproduced on 2 JSBSim builds

Five things that broke, and what I learned from each

Issue	Root cause	Caught by	Resolution
Heading-wrap discontinuity	Heading crossing 360°/0° produced a 358° error instead of 2°, which would command a hard turn the wrong way.	Reading the very first hands-off trim plot before writing any controller.	Wrapped all heading error to [-180°, 180°) inside the environment, before it reaches any controller.
Deterministic turbulence	JSBSim's gust model defaults to a fixed internal seed, so every training episode received the identical disturbance.	Tracing raw state across different episode seeds and finding identical values.	Explicitly randomized atmosphere/randomseed once per episode reset.
Imitation model loses integral memory	The PID's altitude loop has an integrator; the cloned model sees only instantaneous state, so small regression errors compound over the aircraft's ~30s phugoid mode into a slow, growing oscillation invisible in a 60s eval window.	Extending the evaluation horizon to 120s after the model looked fine at 60s.	Not "fixed" – documented as the core motivation for the safety monitor and for moving to RL / DAgger-style training in v2.
PID fallback mis-trims when engaged mid-flight	The PID captured whatever pitch it saw on its first control call as "level." Correct at a fresh reset; wrong when engaged mid-oscillation by the monitor.	Integration-testing the full monitor+fallback system – invisible to a PID-only unit test.	Environment now computes and exposes true trim pitch; PID takes it explicitly instead of inferring it.
Fallback has no airspeed protection	The PID controls only elevator and aileron. A bank (added drag) plus a headwind gust can bleed airspeed through the 50kt floor with nothing to stop it.	Systematic stress sweep across turbulence severity, independent of the model or monitor.	Documented scope limit: reliable up to severity 2, 60–75% crash rate at severity 3–4. Flagged for a throttle/autothrottle loop in a future iteration.

Validation detail

Stress test (N=20 episodes/condition, 90s each, turbulence ≤ severity 2, off-nominal starts to ±150ft, sensor noise, all combined): mean final altitude tracking error, monitor vs. model alone – large off-nominal start: 18.7ft → 6.2ft; combined stress: 17.1ft → 7.5ft.

Fallback reliability limit, characterized independently of the model/monitor (PID alone, turns + turbulence): 0% crash rate at severity 0–2, climbing to 50–65% at severity 3, 60–75% at severity 4 – reproduced consistently across two different JSBSim builds.

Monitor false-trip rate on the PID itself: 0% across all seven nominal test cases – the monitor never second-guesses a well-behaved controller.

Scope: v1 (this writeup) pairs a PID baseline with a behavior-cloned imitation model. v2 replaces the imitation model with one trained via reinforcement learning, using the same safety monitor and fallback.